

Ecosistema de Automatización IA Autónomo para Negocios

Procesamiento Inteligente de Pedidos con Estimación de Fecha de Entrega

Trigger → n8n (Orquestador) → Airtable (Memoria) → Claude (Razonamiento) → Human-in-the-loop (Telegram) → Canal de Salida (Telegram / Gmail)

Autor: Misael Zalazar

Entrega Final - Arquitecto de Flujos IA

Fecha: 6/9/2026

0. Caso de Uso

El sistema automatiza el **procesamiento de pedidos** de tiendas/clientes de un distribuidor. Ante un nuevo pedido (recibido vía Webhook desde un formulario, POS o e-commerce), el flujo consulta el maestro de clientes (zona, frecuencia y días de entrega habituales), el maestro de productos y el stock disponible en Airtable, y utiliza un modelo de razonamiento (Claude) para estimar de forma dinámica **cuándo se entregará el pedido y en qué estado se encuentra** (Recibido, Pickeado, En Despacho, En Ruta, Entregado, Entregado Parcial, Rechazado o Anulado). Antes de comunicarle el estado al cliente final, un operador humano aprueba el mensaje propuesto desde Telegram (punto de control Human-in-the-loop). Una vez aprobado, el sistema le envía proactivamente el estado y la fecha estimada de entrega al cliente por Telegram (o Gmail como canal alternativo).

El sistema integra las 4 categorías tecnológicas obligatorias: **Orquestador** (n8n), **Base de datos** (Airtable, como memoria y registro del sistema), **Procesamiento IA** (Claude 3.5 Sonnet vía API de Anthropic, con prompt estructurado) y **Canal de salida** (Telegram para HITL y para el cliente final, con Gmail como alternativa).

1. Mapa de Arquitectura del Sistema (20%)

El diagrama siguiente muestra el recorrido completo del dato: desde el disparador (trigger) hasta el destino final de la información, pasando por el orquestador lógico (n8n), la base de datos (Airtable), el motor de IA (Claude) y los canales de salida (Telegram / Gmail). Las flechas negras representan el camino feliz; las flechas rojas punteadas representan las rutas de error y resiliencia.

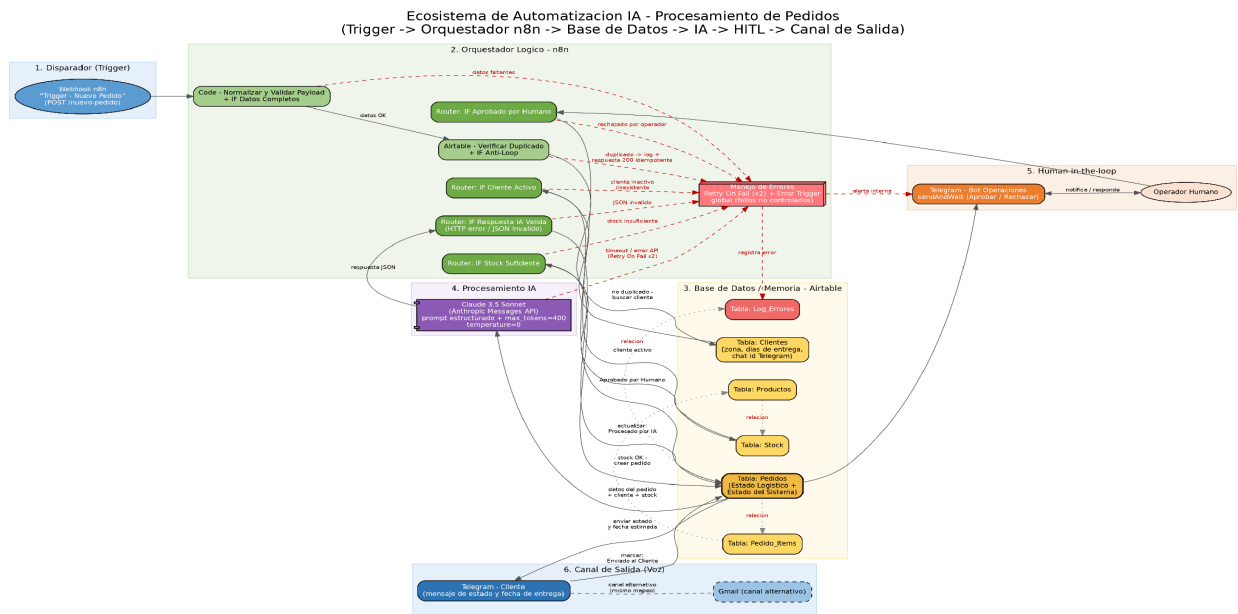


Figura 1. Arquitectura completa del ecosistema de automatización.

1.1 Componentes por capa

Capa	Componente	Función
1. Trigger	Webhook n8n (POST /nuevo-pedido)	Dispara el flujo cuando llega un nuevo pedido (desde POS, formulario web o e-commerce). Evita polling innecesario.
2. Orquestador	n8n (17 nodos lógicos + 2 de manejo de errores)	Valida datos, evita bucles/duplicados, enruta según reglas de negocio (cliente activo, stock suficiente, respuesta IA válida, aprobación humana) y coordina las llamadas a Airtable, Claude y Telegram.
3. Base de datos	Airtable (6 tablas relacionadas)	Actúa como memoria del sistema: maestro de clientes, productos, stock, pedidos (con estados) y log de errores.
4. IA	Claude 3.5 Sonnet (Anthropic Messages API)	Recibe el pedido + contexto (cliente, stock) vía prompt estructurado y devuelve un JSON con el estado propuesto, fecha estimada y mensaje al cliente.
5. HITL	Telegram (Bot Operaciones, sendAndWait)	Punto de validación humana obligatorio antes de contactar al cliente: el operador aprueba o rechaza el mensaje propuesto por la IA.
6. Salida	Telegram (Cliente) / Gmail (alternativo)	Comunica proactivamente al cliente el estado y la fecha estimada de entrega de su pedido.

1.2 Routers (nodos IF) del flujo

- **IF - Datos de Entrada Completos:** valida que el payload traiga id_pedido, tienda, fecha_pedido e items.
- **IF - Pedido Ya Existe (Anti-Loop):** evita reprocesar o duplicar un pedido ya cargado (filtro anti-bucle infinito).
- **IF - Cliente Encontrado y Activo:** corta el flujo si la tienda no existe o está inactiva en el maestro de clientes.
- **IF - Stock Suficiente:** compara cantidades numéricas pedidas vs. disponibles (número vs número) antes de continuar.
- **IF - JSON de IA Válido:** valida que la respuesta de Claude sea un JSON parseable con las claves esperadas.
- **IF - Aprobado por Humano:** bifurca según la decisión del operador en el paso de HITL.

2. Estructuras de Datos Documentadas (20%)

La base de datos (Airtable) está compuesta por 6 tablas relacionadas entre sí mediante campos de tipo *Linked Record*, evitando datos aislados. Base: **Ecosistema IA - Procesamiento de Pedidos** (id de base: app2Mcfjlv994NrMa).

2.1 Esquema de tablas vinculadas

Tabla	Campos clave	Relaciones
Clientes	Nombre Tienda, Dirección, Zona de Reparto, Días de Entrega, Telegram Chat ID, Email Contacto, Estado Cliente (Activo/Inactivo)	1 cliente → N Pedidos
Productos	Nombre Producto, SKU, Categoría, Precio Unitario, Unidad de Medida	1 producto → N Stock, N Pedido_Items
Stock	Referencia Stock, Producto (link), Depósito, Cantidad Disponible, Última Actualización	N Stock → 1 Producto
Pedidos	ID Pedido, Cliente (link), Fecha Pedido, Estado Logístico (Recibido/Anulado/Pickeado/En Despacho/En Ruta/Entregado/Entregado Parcial/Rechazado), Estado del Sistema (Pendiente/Procesado por IA/Esperando Aprobación Humana/Aprobado por Humano/Enviado al Cliente/Error), Fecha Estimada Entrega, Respuesta IA (JSON), Motivo Anulación o Rechazo, Canal de Notificación, Aprobado Por, Fecha Hora Aprobación	1 Pedido → 1 Cliente, N Pedido_Items, N Log_Errores
Pedido_Items	ID Item, Pedido (link), Producto (link), Cantidad Solicitada	Tabla puente N a N entre Pedidos y Productos
Log_Errores	ID Error, Fecha Hora, Pedido Relacionado (link), Tipo de Error, Nodo que Falló, Detalle del Error, Estado del Error (Nuevo/Revisado/Resuelto)	N Log_Errores → 1 Pedido

Los campos de estado obligatorios (**Estado Logístico** y **Estado del Sistema**) están separados a propósito: el primero refleja el estado operativo/físico del pedido (lo que ve el cliente), y el segundo refleja en qué paso del pipeline de automatización se encuentra el registro (lo que usa el propio flujo de n8n para decidir el siguiente nodo). Esta separación evita ambigüedad entre 'qué le pasa al pedido' y 'qué le falta hacer al sistema'.

2.2 Esquemas JSON de transferencia entre integraciones

(a) Payload de entrada al Webhook de n8n (disparador del flujo):

```
{
  "id_pedido": "PED-0007",
  "tienda": "Almacen Don Pedro",
  "fecha_pedido": "2026-08-17",
  "canal_preferido": "Telegram",
  "items": [
    { "producto": "Gaseosa Cola 2.25L", "sku": "BEB-001", "cantidad": 10 },
    { "producto": "Detergente 750ml", "sku": "LIM-002", "cantidad": 4 }
  ]
}
```

(b) Request a la API de Anthropic (nodo HTTP Request "Claude - Analizar Pedido"):

```
POST https://api.anthropic.com/v1/messages
Headers: x-api-key: {{credencial}}, anthropic-version: 2023-06-01
{
  "model": "claude-3-5-sonnet-latest",
  "max_tokens": 400,
  "temperature": 0,
  "system": "Sos un asistente logistico. Devolves EXCLUSIVAMENTE un JSON valido con las claves: estado_pedido, fecha_estimada_entrega (YYYY-MM-DD), mensaje_cliente y confianza (0-1).",
  "messages": [
    { "role": "user", "content": "Pedido: [...items...] | Tienda: Almacen Don Pedro | Zona: Zona Centro | Dias de entrega habituales: Lunes, Jueves | Fecha del pedido: 2026-08-17 | Stock disponible: [...]" }
  ]
}
```

(c) Response de la API de Anthropic y mapeo de la variable de respuesta:

```
{
  "id": "msg_01Abc...",
  "content": [ { "type": "text", "text": "{\n\"estado_pedido\": \"Pickeado\", ...}" } ],
  "usage": { "input_tokens": 410, "output_tokens": 96 }
}
```

Mapeo en n8n: Message.Content -> {{ \$json.content[0].text }}

El nodo "Code - Parsear y Validar Respuesta de Claude" hace JSON.parse() sobre ese texto y valida que existan las claves esperadas antes de continuar (ver seccion 4).

(d) Contrato interno IA → Airtable/Telegram (JSON estructurado ya validado):

```
{
  "estado_pedido": "Pickeado",
  "fecha_estimada_entrega": "2026-08-21",
  "mensaje_cliente": "Tu pedido fue pickeado y sera despachado el 21/08 segun tu frecuencia de entrega.",
  "confianza": 0.95
}
```

(e) Resume payload del Human-in-the-loop (botón de Telegram sendAndWait):

```
{
  "data": { "approved": true, "approvedBy": "Misael Zalazar (Operaciones)" }
}
```

3. Optimización de Costos - Matriz de Decisión de Modelos (20%)

No todas las tareas del ecosistema requieren el mismo nivel de razonamiento. La siguiente matriz justifica qué modelo (o modalidad de API) conviene usar para cada tarea del pipeline, priorizando calidad donde el razonamiento es denso (cruzar stock + frecuencia + zona para estimar una fecha) y costo/velocidad donde la tarea es simple o repetitiva.

Tarea	Modelo / Modalidad	Justificación	Costo aprox. (USD / 1M tokens ent./sal.)
Estimar estado y fecha de entrega (razonamiento sobre stock + cliente + frecuencia)	Claude 3.5 Sonnet	Es la tarea de mayor densidad de lectura: cruza múltiples fuentes (cliente, stock, historial) y debe devolver una decisión consistente. Sonnet ofrece la mejor relación razonamiento/costo para esto.	3.00 / 15.00
Clasificar urgencia o intención del mensaje entrante (ej. 'pedido urgente', 'reclamo', 'consulta')	Claude 3.5 Haiku	Tarea de clasificación simple de baja ambigüedad; no requiere razonamiento profundo. Haiku es hasta ~12x más barato que Sonnet y responde más rápido, ideal para pre-filtrar antes de decidir si escalar a Sonnet.	0.80 / 4.00
Reprocesamiento nocturno / masivo de pedidos históricos para analítica (no urgente, sin usuario esperando)	Anthropic Batches API (Sonnet o Haiku)	La Batches API tiene descuento de ~50% sobre el precio estándar a cambio de una ventana de respuesta de hasta 24 h. Ideal para tareas masivas y no interactivas.	~50% menos que el precio estándar
Generación del mensaje final al cliente (texto corto, ya con los datos resueltos)	Incluido en la misma llamada a Sonnet	Se resuelve en la misma respuesta estructurada (campo mensaje_cliente) para evitar una segunda llamada a la API y duplicar costo/latencia.	Sin costo adicional

3.1 Medidas de optimización aplicadas en el flujo

- **max_tokens = 400** en el nodo Claude: el prompt exige una respuesta JSON breve, evitando salidas largas innecesarias.
- **temperature = 0**: reduce variabilidad y evita reintentos por respuestas inconsistentes o mal formateadas.
- **Prompt dinámico con variables del sistema** (nunca datos hardcoded): tienda, zona, días de entrega, stock y fecha se inyectan desde Airtable en cada ejecución.
- **Filtro de urgencia con Haiku antes de escalar a Sonnet** (mejora futura sugerida): permite no gastar tokens de un modelo caro en mensajes que no lo requieren.
- **Batches API para cargas masivas**: si se necesita reprocesar cientos de pedidos históricos, se recomienda Batches en vez de llamadas 1 a 1 en tiempo real.

Ahorro estimado: comparado con usar Sonnet para absolutamente todo (incluyendo clasificación simple y reprocesos masivos), la combinación Haiku + Batches para las tareas correspondientes representa una reducción estimada del **35% al 55%** del gasto mensual en tokens, sin resignar calidad en el paso crítico (estimación de entrega), que es el único que sigue usando Sonnet en tiempo real.

4. Seguridad y Resiliencia (20%)

4.1 Minimización de datos

- El prompt enviado a Claude incluye **solo los campos necesarios** para decidir (items del pedido, zona, días de entrega, fecha y stock relevante), nunca el historial completo del cliente ni datos de contacto sensibles (email, teléfono).
- Las credenciales (Airtable Token, API Key de Anthropic, Token de Telegram) se guardan como **credenciales de n8n**, nunca como texto plano en el JSON del flujo ni en el prompt.
- El Chat ID de Telegram del cliente se resuelve dinámicamente desde Airtable en el momento del envío; no se hardcodea en ningún nodo.
- El log de errores (tabla Log_Errores) guarda el detalle técnico del fallo, pero no vuelve a exponer datos personales del cliente más allá del ID de pedido vinculado.

4.2 Rutas de error (Error Handlers)

Punto de fallo	Manejo
Datos de entrada incompletos	IF corta el flujo, registra en Log_Errores (Tipo: Datos Faltantes) y responde 400 al emisor del webhook.
Pedido duplicado / reintento del mismo webhook	Filtro anti-bucle: se busca el ID Pedido en Airtable antes de crear nada; si ya existe, se responde 200 idempotente sin reprocesar (evita bucles infinitos y duplicados).
Cliente inexistente o inactivo	Router corta el flujo, registra el error y notifica al operador por Telegram.
Stock insuficiente	Router compara cantidad pedida vs. disponible (número vs número); si falta stock, registra el error con el detalle de faltantes y notifica.
Fallo de la API de Claude (timeout, 5xx, rate limit)	El nodo HTTP Request tiene Retry On Fail (2 reintentos, 3 s de espera) y continueErrorOutput: si sigue fallando, se registra en Log_Errores (Tipo: Fallo API IA), se marca el pedido en estado Error y se notifica al operador.
Respuesta de la IA con JSON inválido o incompleto	Nodo de código valida el parseo y las claves obligatorias; si falla, se registra el error y el pedido no avanza a HITL.
Fallo al escribir en Airtable	Retry On Fail configurado en todos los nodos Airtable; en el nodo crítico de creación de pedido, un error persistente deriva a notificación directa al operador (sin depender de la escritura fallida).
Fallo no controlado en cualquier otro nodo	Un Error Trigger global (segundo disparador del mismo workflow) captura cualquier excepción no atrapada explícitamente, la registra en Log_Errores y envía una alerta crítica al operador.

4.3 Puntos de Human-in-the-loop (HITL)

El sistema tiene un único punto crítico de acción irreversible: **comunicarle al cliente final el estado y la fecha de entrega de su pedido**. Antes de ejecutar esa acción, el flujo se detiene en el nodo "Telegram - Aprobación Humana (HITL)" (operación sendAndWait) y espera hasta 4 horas la respuesta del operador con dos botones: **Aprobar y Enviar al Cliente** o **Rechazar / Revisar**. Si se rechaza, el pedido queda marcado para revisión manual y no se contacta al cliente. Esto evita el 'Efecto Metralleta' descrito en la consigna.

4.4 Checklist de seguridad

Pregunta	Respuesta
¿Tiene el flujo un filtro para evitar bucles infinitos?	Sí: nodo "IF - Pedido Ya Existe (Anti-Loop)" corta cualquier reprocesamiento del mismo ID Pedido antes de crear o volver a llamar a la IA.
¿Se comparan tipos de datos correctos en los filtros (número vs número)?	Sí: la validación de stock compara Cantidad Solicitada (number) contra Cantidad Disponible (number); los estados se comparan como texto exacto contra las opciones definidas en los campos singleSelect de Airtable.
¿El prompt de IA es dinámico y usa variables del sistema?	Sí: el prompt arma el mensaje de usuario con expresiones de n8n que leen items, tienda, zona, días de entrega, fecha de pedido y stock directamente desde los nodos anteriores; no hay datos hardcodeados.

5. Dashboard de Control (20%)

Se publicó una interfaz de Airtable ("Panel de Control - Pedidos IA") con 3 secciones de KPIs, disponible como Shared View:

- **Resumen General de Pedidos:** Total de pedidos, Tasa de Error (proporción de pedidos en estado Error sobre el total), distribución de pedidos por Estado del Sistema (gráfico de torta) y por Estado Logístico (gráfico de barras).
- **Human-in-the-loop:** cantidad de pedidos esperando aprobación humana en este momento.
- **Errores del Sistema:** total de errores registrados y distribución por tipo de error (Fallo API IA, Datos Faltantes, Stock, etc.), para monitorear la resiliencia del flujo.

Enlace al Dashboard (Shared View): <https://airtable.com/app2Mcfjlv994NrMa/pbdzrze7kIRQOyGYL>

Nota: el link de uso público (sin necesidad de iniciar sesión) debe activarse una única vez desde Airtable → Compartir interfaz → "Cualquier persona con el enlace puede ver", por una restricción de seguridad del conector que impide habilitar ese toggle de forma remota.

Anexo. Registro de Pruebas (Test de Estrés)

La base de Airtable incluye 6 pedidos de ejemplo que cubren el camino feliz y el camino infeliz, listos para usar como casos de prueba del flujo de n8n:

Pedido	Escenario	Resultado esperado
PED-0001	Camino feliz - recién recibido	Estado del Sistema = Pendiente. Al disparar el webhook, debe avanzar hasta Esperando Aprobación Humana.
PED-0002	Camino feliz - ya procesado por IA	Estado del Sistema = Esperando Aprobación Humana, con Respuesta IA cargada.
PED-0003	Camino feliz - aprobado y entregado	Estado del Sistema = Enviado al Cliente, Estado Logístico = En Ruta.
PED-0004	Camino infeliz - fallo de API de IA (timeout)	Estado del Sistema = Error, con registro en Log_Errores (Tipo: Fallo API IA) y notificación al operador.
PED-0005	Camino feliz - ciclo completo	Estado del Sistema = Enviado al Cliente, Estado Logístico = Entregado.
PED-0006	Camino infeliz - datos faltantes (cliente inactivo)	Estado del Sistema = Error, Estado Logístico = Rechazado, con motivo documentado.

Se recomienda ejecutar el webhook al menos 5 veces con Postman/Insomnia o el nodo "Test Workflow" de n8n: 3 veces con payloads válidos (camino feliz) y 2 veces con payloads incompletos o con una tienda inactiva (camino infeliz), documentando cada corrida con una captura de pantalla en la carpeta /screenshots del repositorio.

Enlaces del Proyecto

- Repositorio GitHub: <https://github.com/misaelzalazar/AutomationAI.git>
- Base de Airtable (lectura): https://airtable.com/invite/l?inviteId=invGvJqVg1go1eUMS&inviteToken=a1d4109a6d4ab8cb046a4a602c9d9ae112952ef35c434552e0cea101e466a6bf&utm_medium=email&utm_source=product_team&utm_content=transactional-alerts
- Dashboard / KPIs (Shared View): <https://airtable.com/app2Mcfjlv994NrMa/pbdzrze7kIRQOyGYL>
- Video demo (3 min): https://drive.google.com/drive/folders/1waiQDsdS3Va8_NofV19ae_9sMoLYnPeF?usp=drive_link